

20 API Concepts

Complete Developer Notes

A practical beginner-to-intermediate guide to understanding APIs

(No fluffy filler. Just the concepts you need to know.)



What is an API? (Quick Overview)

An API (Application Programming Interface) is a set of rules that allows different software applications to communicate with each other.

Client (App/Browser) ↔ API (The Waiter) ↔ Server/Database (The Kitchen)

1. ENDPOINT

Definition: An API endpoint is a specific URL through which a client accesses a particular resource or operation provided by an API.

What it does / How it works:

The client sends an HTTP request to an endpoint. The server processes the request and returns a response.

Why it is used:

Endpoints organize API functionality into specific locations for resources such as users, products, orders, or payments.

Real-world example:

In an online plant shopping app (like Verdora), an endpoint fetches the catalog of available plants or submits a new order.

Technical example:

```
GET /api/plants
GET /api/plants/402
POST /api/orders
```

`/api/plants` represents a collection of plants, while `/api/plants/402` represents one specific plant.

Key points:

- Endpoint = specific API access point.
- Usually consists of a URL/path.
- Different endpoints expose different resources or operations.
- HTTP methods determine what action is performed.

[Client] — HTTP Request —> `/api/plants` —> [API Server] — HTTP Response —> [Client]

2. HTTP METHODS

Definition: HTTP methods (or verbs) indicate the desired action to be performed on the given resource.

What it does / How it works:

When a client calls an endpoint, it attaches an HTTP method to tell the server exactly what it wants to do with the data.

Why it is used:

Provides a standardized way to communicate intent without needing unique URLs for every single action.

Real-world example:

In a social media app, getting your feed is a GET, posting a photo is a POST, updating your bio is a PATCH, and removing a comment is a DELETE.

Technical example:

Method	Endpoint	Action
GET	/users	Retrieve users
POST	/users	Create a user
PUT	/users/101	Replace user 101
PATCH	/users/101	Partially update user 101
DELETE	/users/101	Delete user 101

Key points:

- HTTP methods communicate the intended operation.
- GET is generally used for retrieval.
- POST commonly creates resources.
- PUT generally replaces a resource, while PATCH partially updates it.

GET (Read) | POST (Create) | PUT (Replace) | PATCH (Modify) | DELETE (Remove)

3. REQUEST & RESPONSE

Definition: An API request is what the client sends to the server. An API response is what the server sends back.

What it does / How it works:

A **Request** contains: URL, HTTP method, Headers, Query/Path parameters, and an optional Body.
A **Response** contains: Status code, Headers, and a Response body.

Why it is used:

This request-response cycle is the fundamental communication pattern of the web.

Real-world example:

When you open a weather app, your phone sends a request for your city. The server responds with the temperature and forecast data.

Technical example:

Request: `GET /api/users/101`

Response:

```
{
  "id": 101,
  "name": "Alex",
  "email": "alex@example.com"
}
```

Key points:

- Client sends request; Server processes it and returns response.
- Headers carry metadata (like auth tokens or content types).
- JSON is commonly used for structured API data bodies.

 Client $\longrightarrow$ (Method + URL + Headers + Body) $\longrightarrow$  Server

 Client $\longleftarrow$ (Status Code + Headers + Body) $\longleftarrow$  Server

4. HTTP STATUS CODES

Definition: Status codes communicate the result of an HTTP request. They tell the client if the request succeeded, failed, or requires further action.

What it does / How it works:

The server attaches a 3-digit number to every response. The first digit defines the category of the outcome.

Why it is used:

Provides a standardized, machine-readable way for clients to handle success and error states.

Real-world example:

Submitting a payment form successfully yields a 200/201. If the product is out of stock (not found), you might get a 404.

Technical example:

- **1xx** — Informational
- **2xx** — Success (200 OK, 201 Created, 204 No Content)
- **3xx** — Redirection (301 Moved Permanently, 304 Not Modified)
- **4xx** — Client Error (400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 429 Too Many Requests)
- **5xx** — Server Error (500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable)

Key points:

- **401** = authentication is missing/invalid (Who are you?)
- **403** = identity is known, but access is not allowed (You don't have permission).
- 4xx means the client messed up; 5xx means the server messed up.

1xx (Info) ↗ 2xx (Success!) ↗ 3xx (Redirect) ↘ 4xx (Your fault) ↘ 5xx (Server's fault)

5. AUTHENTICATION

Definition: Authentication verifies the identity of a user or system. It answers the question: "Who are you?"

What it does / How it works:

A client provides proof of identity (credentials) to an authentication server, which validates them and issues a token or session.

Why it is used:

To protect user accounts, verify clients, and prevent unauthorized identity claims.

Real-world example:

Typing your email and password into a banking app, or using FaceID to unlock it.

Technical example:

Common methods: Username + password, API keys, Bearer tokens, OAuth-based login.

Key points:

- Authentication = identity verification.
- It happens *before* deciding what the user is allowed to access.
- Credentials/tokens should be protected. Never expose sensitive secrets.

User — (Credentials) —> Auth Server — (Verified Identity / Token) —> API

6. AUTHORIZATION

Definition: Authorization determines what an authenticated user or system is allowed to access or perform. It answers: "What are you allowed to do?"

What it does / How it works:

After a user is authenticated, the system checks their permissions, roles, or scopes before granting access to a specific resource.

Why it is used:

Ensures that users can only access their own data and that only admins can perform sensitive actions.

Real-world example:

In a blog platform:

Admin → can delete any user's post.

Editor → can edit content.

Viewer → can only read content.

Key points:

- Authentication verifies identity; Authorization determines permissions.
- Can use roles, permissions, scopes, policies, etc.
- A user can be authenticated but still unauthorized for a specific action (yielding a 403 Forbidden).

Login (AuthN) —> User Identity —> Permission Check (AuthZ) —> Resource Access

7. ACCESS TOKENS

Definition: An access token is a credential that a client presents to an API to access protected resources.

What it does / How it works:

Instead of sending a username and password with every request, the client sends a token (often a JWT - JSON Web Token). The API validates this token before granting access.

Why it is used:

Tokens are temporary, secure, and can be scoped to specific permissions without exposing base credentials.

Real-world example:

Like a hotel key card: you prove who you are at the front desk (Authentication), they give you a key card (Access Token), and the card lets you into specific rooms (Authorization).

Technical example:

```
GET /api/profile
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR...
```

Key points:

- **Access token** → used to access protected resources.
- **Refresh token** → used to obtain a new access token when the current one expires.
- Treat access tokens as sensitive credentials; do not expose them.
- APIs commonly receive bearer tokens through the Authorization header.

Client — (Access Token) —> API — (Token Validation) —> Protected Resource

8. OAUTH 2.0

Definition: OAuth 2.0 is an authorization framework that allows applications to obtain limited access to protected resources without requiring the user to share their password with the application.

What it does / How it works:

It delegates authentication to an identity provider and issues access tokens with specific "scopes" (permissions) to third-party clients.

Why it is used:

Allows secure, scoped access to APIs on behalf of a user.

Real-world example:

Clicking "Sign in with Google" on a new app, and seeing: "Allow this application to access your profile information and contacts."

Key points:

- OAuth 2.0 is primarily about authorization.
- It involves: Resource Owner (User), Client (App), Auth Server, and Resource Server (API).
- Uses tokens and scopes; the user never gives their password to the third-party client.

User —> Client App —> Auth Server — (Grants Permission) —> Client App gets Token —> API

9. RATE LIMITING

Definition: Rate limiting restricts how many requests a client can make to an API within a specified period.

What it does / How it works:

The server tracks the number of requests from a specific IP, API key, or user token. If the limit is breached, the server blocks further requests for that time window.

Why it is used:

Prevent abuse (like brute-force attacks), protect infrastructure, maintain fair usage among all users, and control traffic spikes.

Real-world example:

A free-tier weather API allows only 100 requests per minute per API key. Exceeding this returns an error.

Technical example:

Response when exceeded: `429 Too Many Requests`

Key points:

- Limits request frequency based on a quota.
- Based on identifiers (IP, API key, user ID).
- APIs often communicate limits via headers (e.g., `X-RateLimit-Remaining`).

Req 1 —> Req 2 —> Req 3 ... —> Limit Reached  (429 Too Many Requests)

10. THROTTLING

Definition: Throttling controls the rate at which requests are processed or accepted, especially when traffic becomes excessive.

What it does / How it works:

Instead of just counting a quota (Rate Limiting), throttling regulates the actual flow of traffic into the system, sometimes slowing down responses or temporarily rejecting requests to keep the server alive.

Why it is used:

Protects system resources, smooths out sudden traffic spikes, and ensures the API doesn't crash under heavy load.

Real-world example:

During a massive Black Friday sale, an e-commerce API might throttle incoming checkout requests—placing them in a queue or slowing them down—so the database doesn't crash.

Key points:

- **Rate limiting:** defines a request allowance/quota (e.g., 100/min).
- **Throttling:** dynamically controls traffic processing when the system needs to regulate load.
- Like a water pipe valve, throttling restricts flow to prevent bursts.

 Huge Traffic —> [Throttling Valve] —>  Controlled Request Flow —> API Server

11. PAGINATION

Definition: Pagination divides a large dataset into smaller pages instead of returning everything in one massive response.

What it does / How it works:

Clients specify which subset of data they want using query parameters (like page number and limit, or a cursor).

Why it is used:

Reduces response payload size, improves server performance, and makes large datasets manageable for clients.

Real-world example:

Scrolling through hundreds of plants in the Verdora app. The app fetches 20 plants at a time rather than downloading the entire database at once.

Technical example:

Offset/Page-based: `GET /api/products?page=2&limit=20`

Cursor-based: `GET /api/products?cursor=abc123&limit=20`

Key points:

- Essential for large collections (feeds, search results).
- Offset/page-based and cursor-based are the two most common approaches.
- Cursor-based is often preferred for real-time feeds to prevent duplicate items when data changes.

1000 Items total —> Page 1 (20) | Page 2 (20) | Page 3 (20) ...

12. CACHING

Definition: Caching stores frequently requested data temporarily so future requests can be served faster without repeatedly performing the same expensive operation.

What it does / How it works:

When a client requests data, the API checks the cache. If present (cache hit), it returns it instantly. If missing (cache miss), it fetches from the database, stores a copy in the cache, and returns it.

Why it is used:

Reduce latency, decrease database workload, improve scalability, and speed up repeated requests.

Real-world example:

A food delivery app caches the list of available restaurants in your area for 5 minutes, as they don't change every second.

Key points:

- Cached data can become stale.
- Cache expiration (TTL - Time To Live) controls how long data remains cached.
- Cache invalidation (removing outdated data) is one of the hardest problems in computer science.
- Not every piece of data should be cached (e.g., live bank balances).

Hit: Client —> API —> Cache —> Client
Miss: Client —> API —> Cache —> Database —> Cache —> Client

13. IDEMPOTENCY

Definition: An operation is idempotent when making the same request multiple times has the same intended effect on the resource as making it once.

What it does / How it works:

If you retry a failed request due to a network timeout, an idempotent API ensures you don't accidentally create duplicate records or charge a credit card twice.

Why it is used:

Helps safely retry requests in distributed systems where network failures cause uncertain outcomes.

Real-world example:

If you click "Pay" and the network drops, your app retries the payment request. With an idempotency key, the server recognizes it as the same transaction and doesn't charge you a second time.

Technical example:

`PUT /users/101` (Setting the name to "Alex" repeatedly leaves the state the same).

Using a key: `Idempotency-Key: abc-123` in the request headers.

Key points:

- Same intended operation = same resulting resource state.
- HTTP GET, PUT, DELETE are generally idempotent. POST is typically not.
- "Idempotent" does not mean the HTTP response code will be identical every time, just the resource state.

Request 1 —> Server (Resource Updated)

Request 1 (Retry) —> Server (Resource state remains the same)

14. WEBHOOKS

Definition: A webhook is a mechanism where one system sends an HTTP request to another system automatically when a specific event occurs.

What it does / How it works:

Instead of the client constantly asking the server "Is it done yet?" (Polling), the client provides a URL, and the server pushes data to that URL via POST when the event happens.

Why it is used:

Provides real-time event notifications and avoids unnecessary polling, saving server resources.

Real-world example:

When a Stripe payment succeeds, Stripe sends a webhook to your backend (e.g., `/webhooks/payment`) so you can update the order status to "Paid".

Key points:

- Event-driven communication.
- Usually uses HTTP POST callbacks.
- Webhook endpoints should validate incoming requests (e.g., verifying a signature) to ensure they actually came from the trusted provider.

Polling: Client — "New?" —> API (No) | Client — "New?" —> API (Yes)
Webhook: Event Occurs —> Service automatically sends HTTP POST —> Your Server

15. API VERSIONING

Definition: API versioning allows an API to evolve and introduce breaking changes while maintaining compatibility for existing clients.

What it does / How it works:

APIs are labeled with a version number. When major structural changes are made, a new version is created, leaving the old one intact for a period.

Why it is used:

Protects existing mobile apps or external partners from instantly breaking when you update the API response structure.

Real-world example:

v1 returns `{ "name": "Alex" }`. v2 changes it to `{ "firstName": "Alex", "lastName": "Smith" }`. An app relying on `name` would crash without versioning.

Technical example:

Common approaches:

URL: `/api/v1/users` vs `/api/v2/users`

Header: `Accept: application/vnd.myapi.v2+json`

Key points:

- Protects clients from breaking changes.
- Provides a controlled migration path.
- Deprecated versions should have a clear sunset/migration plan.

[v1] — stable —> deprecated —> sunset
[v2] — current —> future standard

16. OPENAPI

Definition: OpenAPI is a standard specification for describing HTTP APIs in a machine-readable format.

What it does / How it works:

You write a YAML or JSON file detailing all your endpoints, HTTP methods, parameters, request bodies, and expected responses. Tools read this file to understand the API.

Why it is used:

Generates interactive documentation, auto-generates client SDKs, configures API gateways, and allows automated testing.

Real-world example:

Using Swagger UI (built from OpenAPI specs) to browse an API's documentation and click "Try it out" to send real requests from the browser.

Technical example:

```
paths:
  /users:
    get:
      responses:
        '200':
          description: Successful response
```

Key points:

- OpenAPI = the specification standard. (Swagger = the older name and common tooling).
- Makes API contracts explicit.
- OpenAPI *describes* the API; it does not implement it.

OpenAPI Spec —> Documentation | Client SDK Generation | Automated Testing

17. REST VS GraphQL

REST

Resource-oriented API style using HTTP semantics and multiple endpoints.

Example:

```
GET /users/101
```

```
GET /users/101/orders
```

- Multiple endpoints.
- Uses standard HTTP methods naturally.
- Simple, predictable, widely adopted.
- Trade-off: Can cause over-fetching (getting more data than needed) or under-fetching.

GraphQL

A query language where clients explicitly request exactly the fields they need, usually via one endpoint.

Example:

```
query { user(id: 101) { name email } }
```

- Single endpoint (usually `/graphql`).
- Client controls response payload.
- Strongly typed schema.
- Trade-off: Harder to cache at the HTTP level; complex queries can impact server performance.

Key points:

- Neither is universally "better"; choice depends on application requirements.
- REST and GraphQL solve API design problems differently.

REST: Client —> [Endpoint A], [Endpoint B], [Endpoint C]
GraphQL: Client —(Query for specific fields)—> [Single Endpoint]

18. API GATEWAY

Definition: An API Gateway is a server that acts as a single entry point between clients and multiple backend services.

What it does / How it works:

Instead of clients calling 10 different microservices directly, they call the API Gateway. The gateway routes the request to the correct backend service.

Why it is used:

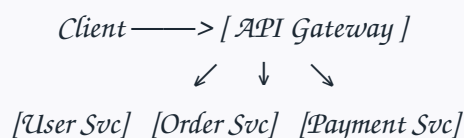
Centralizes cross-cutting concerns: routing, authentication, rate limiting, logging, and request transformation.

Real-world example:

A mobile app sends a request to `api.example.com`. The gateway checks the auth token, then silently routes the request to the hidden internal Payment Service.

Key points:

- Gateway is a manager, not the API logic itself.
- Common in distributed/microservices architectures.
- Avoid putting heavy business logic inside the gateway.



19. MICROSERVICES

Definition: Microservices architecture structures an application as multiple relatively small, independently deployable services organized around business capabilities.

What it does / How it works:

Instead of one massive monolithic codebase, the system is broken into separate services that communicate via APIs or events.

Why it is used:

Allows independent deployment, independent scaling, separate team ownership, and isolation of failures.

Real-world example:

An e-commerce platform (like Verdora) split into: User Service, Plant Catalog Service, Order Service, and Payment Service. If the Order Service crashes, users can still browse the catalog.

Key points:

- Microservices are an architectural style, not just "many APIs".
- Trade-offs: Increases operational complexity, network latency, and distributed debugging difficulty.
- Smaller does not automatically mean better; monoliths are often fine for simpler apps.

API Gateway —> [User Service] [Product Service] [Order Service]

20. ERROR HANDLING

Definition: API error handling is the process of detecting, communicating, logging, and safely responding to failures.

What it does / How it works:

When something goes wrong, the API catches the error and returns a structured, predictable error response along with an appropriate HTTP status code.

Why it is used:

Gives developers enough information to debug the issue without exposing sensitive internal server details to malicious users.

Technical example:

```
{
  "error": {
    "code": "USER_NOT_FOUND",
    "message": "The requested user does not exist."
  }
}
```

Key points:

- Use appropriate HTTP status codes (Don't return 200 OK for a failure!).
- Provide a stable, machine-readable error code.
- NEVER expose passwords, database credentials, or internal stack traces in production API responses.

Request —> API —> Success?
YES —> 2xx Response
NO —> Determine error —> 4xx/5xx Structured JSON Error Response

20 API Concepts — Quick Revision

API Fundamentals

- Endpoint
- HTTP Methods
- Request & Response
- Status Codes

Reliability & Comms

- Idempotency
- Webhooks
- Versioning
- Error Handling

API Security

- Authentication
- Authorization
- Access Tokens
- OAuth 2.0

Architecture & Design

- OpenAPI
- REST vs GraphQL
- API Gateway
- Microservices

Traffic & Performance

- Rate Limiting
- Throttling
- Pagination
- Caching

The Mini Checklist:

- ✓ Understand request → response flow
- ✓ Know GET, POST, PUT, PATCH, DELETE
- ✓ Understand 2xx / 4xx / 5xx differences
- ✓ Know Authentication (Identity) vs Authorization (Permissions)

- ✓ *Understand tokens and basic OAuth concepts*
- ✓ *Know Rate Limiting (Quota) vs Throttling (Flow control)*
- ✓ *Understand pagination and caching benefits*
- ✓ *Understand idempotency and event-driven Webhooks*
- ✓ *Understand why API versioning matters*
- ✓ *Know OpenAPI (Contracts/'Docs')*
- ✓ *Understand REST vs GraphQL trade-offs*
- ✓ *Understand gateways and microservice architecture basics*
- ✓ *Know how APIs should safely handle errors*